

AI Exterior Studio

Final Documentation Pack

AI-Based Exterior House Renovation & Cost Estimation System

This merged PDF contains the same content as the individual chapter PDFs, in order:

#	Document
01	01 — System Overview
02	02 — System Architecture
03	03 — User Workflow
04	04 — Estimation & Costing
05	05 — Limitations
06	06 — API Reference
07	07 — Setup & Deployment

01 — System Overview

1. Problem

Homeowners planning exterior renovation face three barriers before construction:

Gap	Effect
Visualization	They cannot see how <i>their</i> house will look with new cladding, paint, or tiles
Material selection	Choice of finish, location, and durability is guesswork or brochure-based
Cost uncertainty	Quotations vary; area, wastage, and labour are opaque

The product is a **pre-construction planning assistant**: upload a photo, choose finishes, see a redesign of the actual house, and get an approximate quantity and cost breakdown suitable for discussion with contractors.

2. Goals (from problem statement)

1. Accept exterior images of a residential building
2. Generate redesigned visual options using selected materials
3. Estimate material quantities required
4. Calculate a transparent renovation cost

3. Target users

Role	Use
Primary	Homeowners, new house owners before finishing
Secondary	Contractors, architects, builders, consultants, material suppliers

4. Scope of this prototype

In scope

- Single exterior photo upload with quality guidance
- AI segmentation of major facade categories

- Manual mask refinement (brush / eraser)
- Material catalog (paint + textures: cladding, tiles, patterns)
- Classical CV rendering preserving structure and lighting
- Before / after comparison
- Surface quantity from mask coverage $\times$ user facade size
- INR cost estimate with editable rates
- Downloadable PDF report

Out of scope (by design)

- Interior design
- Structural / engineering calculations
- Professional CAD drawings or survey equipment
- Legally binding quotations (estimates are **advisory**)
- Multi-photo fusion, NeRF / full 3D reconstruction
- User authentication / multi-tenant accounts (single-user local demo)

5. Assumptions (aligned with problem statement)

- Residential, low-rise (independent house, bungalow, small apartment)
- Exterior renovation only
- User can approximate facade width and height in metres
- One photo shows the primary elevation of interest
- Cost estimates are for planning discussion, not contracts

6. Constraints met

Constraint	How we satisfy it
No architectural drawings required	Photo-only input
No specialised measuring equipment	User facade W $\times$ H + mask fractions
Rely primarily on visual media	Upload $\rightarrow$ segment $\rightarrow$ render pipeline
Usable by non-technical users	Guided studio UI, category buttons, cost page

Reasonable time on standard connections	CPU classical render (~seconds); segmentation slower on first load
---	--

7. Solution shape (one sentence)

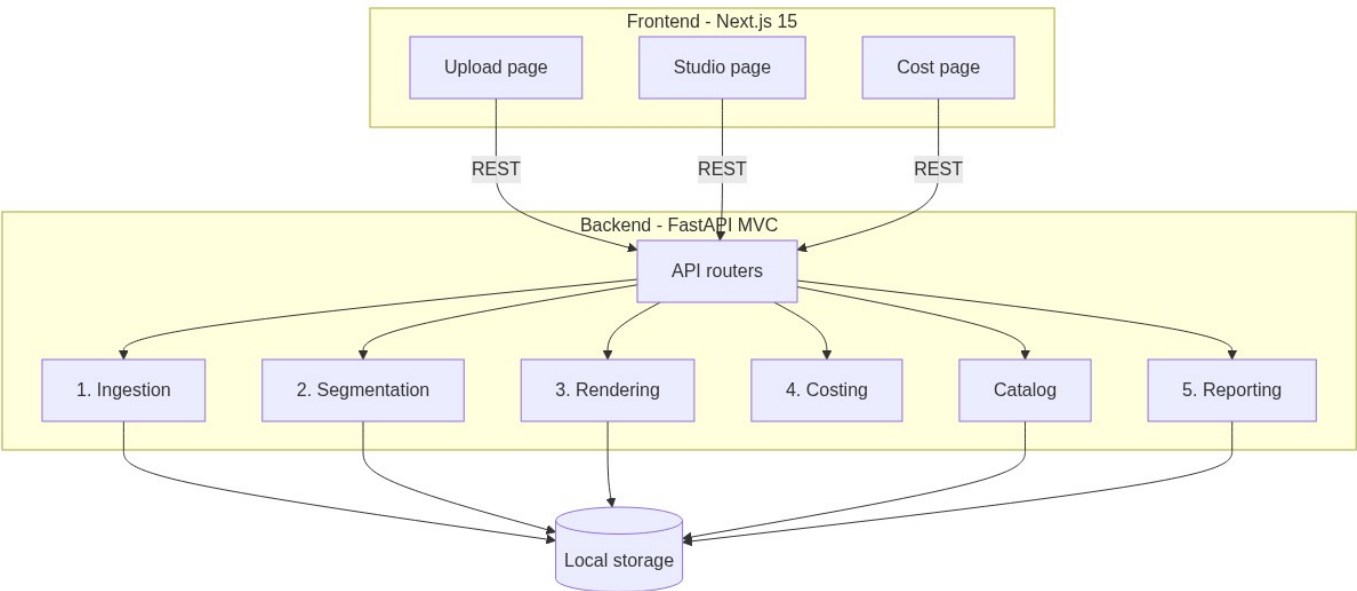
AI perceives regions; deterministic math costs them — segmentation and rendering produce a RegionMap; quantity and cost are pure formulas over that map plus editable rates, never opaque model guesses for money.

02 — System Architecture

Problem-statement deliverable: *A system architecture*

1. High-level architecture

The platform is a **client-server** app with five backend engines glued by a shared **RegionMap** contract. Uploads, masks, renders, textures, and model weights live on **local disk** under `backend/storage/`.



Core idea: RegionMap

After segmentation, each facade category becomes a **region**:

Field	Role
category	wall, balcony, rooftop, gate, window, door, ...
mask	Pixel coverage of that element
polygons	Simplified contours for UI overlay
pixel_area	Count of mask pixels
instance_count	How many discrete detections were merged
confidence	Mean detector score

Rendering, costing, and reporting all **read** this structure. Costing never invents areas — it only scales mask fractions.

2. Backend layers (MVC)

backend/app/ api/ → Controllers (HTTP routers) schemas/ → Request/response DTOs (Pydantic)
services/ → Business logic (engines) core/ → Config, logging utils/ → Categories, image I/O, model paths

Engine	Responsibility	Primary module
Ingestion	Validate, orient, resize, quality warn, save working image	services/ingestion_engine.py
Segmentation	Grounding DINO boxes → SAM masks; wall derivation	services/segmentation/
Rendering	Paint (HSV/LAB shade) or texture tile per region	services/rendering/classical.py
Catalog	Material list from texture folders + default rates	services/catalog.py
Costing	Mask × facade area × rate → INR lines	services/costing.py
Reporting	PDF: before/after, materials, costs	services/report.py

3. Frontend structure

frontend/ app/page.tsx → Upload view app/studio/[projectId]/ → Main studio controller
app/cost/page.tsx → Rate card (INR) components/ → Canvas, mask editor, materials, cost panel, before/after lib/api.ts → Typed REST client

4. Tech stack

Layer	Choice	Rationale
API	FastAPI + Uvicorn	Typed Python AI stack, OpenAPI docs
Frontend	Next.js 15, React 19, TypeScript, Tailwind	App Router UI, canvas workflows
Segmentation	Grounding DINO tiny + SAM vit-base (transformers, CPU)	Zero-shot text categories; no training data

Rendering	OpenCV classical only	Fast, free, structure-preserving on ~8 GB RAM
PDF	ReportLab	Self-contained server-side report
Storage	Local backend/storage/ mounted at /storage	Uploads, masks, renders, textures, weights
Deploy	Docker Compose (recommended)	Fully dockerized local run; ~8 GB RAM for models

Deliberate non-choices for this prototype: GPU/async job queue, Stable Diffusion / ControlNet (memory), full depth/homography survey, authentication. Railway free-tier cloud deploy was evaluated but not used (about 1 GB RAM vs ~8 GB needed — see Doc 07).

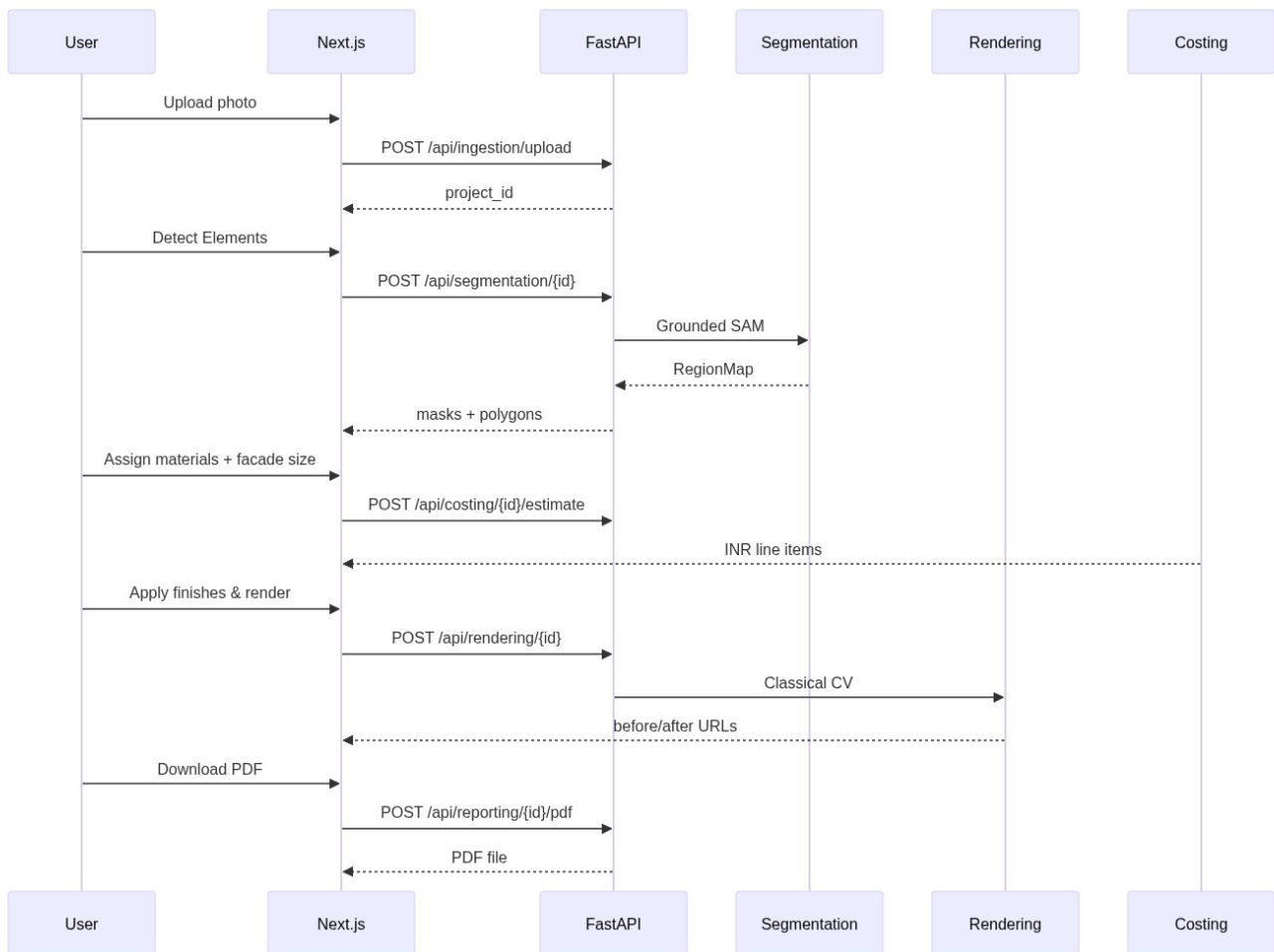
5. Core data structures (in pipeline / API)

Pipeline concepts exchanged via APIs and local files:

Concept	Key fields	Where stored / used
Project	id, filename, image path, width, height, stage	Upload folder + API responses
Region (RegionMap)	category, mask path, polygons, pixel area, instance count, confidence	Mask files under storage/masks/ + segmentation API
Material	key, name, render path, group, unit, rate (INR)	Texture catalog on disk + Cost page rates

Workflow stages for a project photo: **uploaded** → **segmented** → **rendered**.

6. Request sequence (happy path)



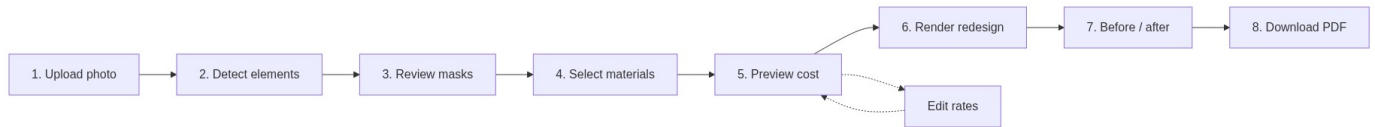
7. Compute notes

- All ML runs **CPU-only** in this build.
- Segmentation is synchronous; first request after process start is slow (model warm-up). Subsequent category runs reuse loaded models.
- Rendering is typically under a few seconds for working resolution (long edge capped, see ingestion settings).
- Costing is pure CPU math and returns in milliseconds.

03 — User Workflow

Problem-statement deliverable: *A user workflow*

1. End-to-end journey



2. Step-by-step

Step 1 — Upload (/)

- User drops or selects a JPEG / PNG / WebP of the house exterior.
- Backend **Ingestion Engine** validates type/size, EXIF-oriens, resizes long edge, and may return **warnings** (blurry, too dark, too bright).
- User is redirected to `/studio/{projectId}`.

Tip for better results: Straight-on facade photo in good daylight; avoid extreme zoom or heavy obstruction.

Step 2 — Detect elements (Studio)

- User clicks **Detect Elements**.
- Default categories: wall, balcony, rooftop, gate, window, door (railing / pillar available in taxonomy).
- Grounding DINO proposes boxes from text prompts; SAM produces masks.
- **Walls** are derived as building silhouette minus openings (more reliable than direct “wall” detection).
- Colored overlays appear on the canvas.

Step 3 — Review / refine masks

- User selects a category and can **brush / erase** the mask (`MaskEditor`) so missed areas or false positives are corrected.
- Changes are saved to the RegionMap via `PUT /api/segmentation/{id}/regions/{category}`.
- This human-in-the-loop step turns an imperfect model into a usable plan.

Step 4 — Select materials

- For each detected category, user picks:
- **Solid paint** + color, or
- A **texture** (wall cladding, wall tiles, texture patterns) from the catalog.

- Selections can be changed and re-rendered.

Step 5 — Preview cost

- Studio shows a live **cost estimate** panel.
- User sets approximate **facade width × height (m)** (defaults 12 m × 9 m).
- Quantities use mask pixel share × facade area × default 10% waste (area materials).
- Gates / doors / windows / pillars price by **piece count**.
- Optional: open **/cost** to edit material rates (INR); estimates recalculate from stored rates.

Step 6 — Render redesign

- User clicks **Apply finishes & render**.
- Classical rendering applies paint (shade-preserving recolor) or tiles textures inside masks, in a fixed layer order (walls first, openings later).
- Output preserves original geometry and approximate lighting.

Step 7 — Before / after

- Side-by-side or slider comparison of original vs redesigned image.
- User may change materials and re-render.

Step 8 — Download report

- After a successful render, user downloads a **PDF** containing:
- Branding / title
- Original and redesigned images
- Selected materials
- Quantity / cost breakdown (INR)
- Advisory disclaimer

The PDF is intended as a **discussion document** with contractors, not a formal quotation.

3. Screens and responsibilities

Screen	Responsibility
/	Capture project photo

/studio/[projectId]	Detect, edit, assign finishes, estimate, render, export
/cost	Maintain editable rate card shared across projects

4. Typical timing (local CPU)

Stage	Expected order of magnitude
Upload + ingest	under 2 s
First segmentation (cold models)	tens of seconds to a few minutes
Later segmentation / same session	shorter (models warm)
Cost estimate	under 1 s
Classical render	a few seconds
PDF	a few seconds

Exact times depend on CPU, RAM, and image size.

04 — Estimation & Costing

Problem-statement deliverable: *Documentation explaining how the estimation works*

Source of truth in code: `backend/app/services/costing.py`.

1. Design principle

Costing is deterministic. Segmentation supplies pixel masks; the user supplies approximate facade dimensions and (optionally) rates. No generative model invents rupees. Every line item is explainable as quantity $\times$ rate.

2. Inputs

Input	Source	Default
Category masks (<code>pixel_area</code> , <code>instance_count</code>)	Segmentation / mask edit	—
Image size (<code>width</code> $\times$ <code>height</code>)	Ingested project image	—
Facade width & height (m)	Studio UI	12 m $\times$ 9 m
Waste factor	Engine constant	1.10 (10%)
Material <code>rate_inr</code> & <code>unit</code>	Material catalog / Cost page	Group defaults if rate $\leq$ 0
Material selections	Studio (category $\rightarrow$ material)	—

3. Surface / quantity model

3.1 Facade area

[
 $A_{\text{facade}} = W_{\text{facade}} \times H_{\text{facade}} \quad (\mathrm{m}^2)$
]

Widths/heights are clamped to a minimum of **0.5 m** to avoid division-by-zero nonsense.

3.2 Area-based materials (`unit == "sqm"`)

Most facade finishes (paint, cladding, tiles, patterns):

[
 $q = \left(\frac{\text{pixel_area}}{\text{image_width} \times \text{image_height}}\right) \times A_{\text{facade}} \times w$
]

where ($w \geq 1$) is the waste factor (default **1.10**).

- If ($0 < q < 0.25$), quantity is raised to **0.25 sqm** (avoids ■0 for tiny valid patches).
- Display unit: **sqm**.

Interpretation: The fraction of the photo covered by the mask is treated as the same fraction of the user-stated facade area. This is approximate and assumes a roughly frontal elevation.

3.3 Piece-based categories (unit == "unit")

Categories: **gate, door, window, pillar**.

```
[
q = \max(1, \; \text{instance\_count})
]
```

Display unit: **pcs**.

3.4 What is *not* computed (yet)

- Paint volume in litres (no coats × coverage curve)
- Tile packing by piece size
- Railing length in running metres
- Separate labour vs material columns

Those are natural extensions of the same RegionMap (see Limitations).

4. Cost formula

For each selection line:

```
[
\text{line\_total} = q \times \text{rate\_inr}
]

[
\text{grand total} = \sum \text{line\_totals}
]
```

Subtotal equals total in the current build (no tax / contingency line beyond the waste factor).

Currency: **INR (■)**. PDF uses **Rs.** for font glyph safety.

5. Default rates (■)

Used when the stored rate is missing or ≤ 0 :

Material group	Default rate	Typical unit
paint	55	■/sqm

cladding	2200	■/sqm
tiles	1100	■/sqm
patterns	1500	■/sqm
texture (fallback)	1200	■/sqm

Users override these on `/cost` via PUT `/api/materials/rates`. Recalculation uses the new rates on the next estimate — no re-segmentation required.

6. Worked example

- Image: 1200 × 900 px → 1 080 000 px
- Facade: 12 m × 9 m → **108 m²**
- Wall mask: 270 000 px → fraction **0.25**
- Material: exterior emulsion (paint) at ■55/sqm
- Waste: 1.10

```
[
q = 0.25 \times 108 \times 1.10 = 29.7\;\mathrm{sqm}
]

[
\text{line} = 29.7 \times 55 = \text{■}1{,}633.50
]
```

Add other category lines (balcony cladding, gate pieces, ...) for the grand total.

7. Live estimate vs PDF

Path	Endpoint	Behaviour
Live studio panel	POST <code>/api/costing/{project_id}/estimate</code>	JSON line items + totals + disclaimer
PDF report	POST <code>/api/reporting/{project_id}/pdf</code>	Same logic embedded with images (requires prior render)

8. Disclaimer (shown in API and PDF)

Approximate estimate based on selected finishes, detected mask coverage, and rates from the Cost page. **Not a formal quotation.**

This matches problem-statement Section 7: cost estimates are **advisory**.

05 — Limitations

Problem-statement deliverable: *Documentation explaining limitations of the system*

Estimates and visuals are planning aids. Treat them as approximate.

1. Advisory costing (explicit)

- Outputs are **not** legally binding quotations or contractor bills.
- Rates are catalogue defaults or user edits — not live market surveys.
- Labour is folded into a single blended `rate_inr` per material, not a separate labour schedule.
- Waste is a flat **10%** factor, not site-specific wastage.

2. Area & quantity accuracy

Limitation	Impact
User must enter facade W×H	Wrong dimensions scale every line item
Mask fraction = area fraction	Perspective / depth / foreshortening not corrected
No door-height or window-scale reference	Automatic metric scale unavailable
Single photo	Only visible surfaces; sides/back omitted
Unit categories use instance counts	Missed or merged detections skew piece counts
No paint L / tile piece / railing RFT packing	Quantities are sqm or pcs only

Improve accuracy by correcting masks carefully and measuring the real facade width/height with a tape.

3. Segmentation

- Grounding DINO can miss unusual architecture, heavy occlusion, night photos, or cluttered scenes.
- Wall quality depends on building box quality and opening subtraction.
- Soft edges / vegetation may bleed into masks — use brush/eraser.
- Parapet may appear as part of wall or rooftop depending on prompt hits.
- CPU-only; **cold start is slow**; unsuitable for high-concurrency production without a job queue.

4. Visualization / rendering

- Rendering is **classical OpenCV** (recolor + texture tiling), not photorealistic diffusion.
- Textures tile approximately; fine mortar joints and specular materials will not match showroom photos.
- Lighting approximation preserves luminance shading but is not full global illumination.
- ControlNet / Stable Diffusion paths exist conceptually in earlier design docs but are **disabled** (memory / RAM budget ~8 GB).
- Structure preservation is strong for paint; texture placement can still look “stamped” on complex geometry.

5. Product / platform gaps

Gap	Note
No authentication	Local single-user demo
No multi-design named variants	Last render overwrites output image
No multi-photo / 360 capture	One elevation at a time
Building-presence rejection soft	Severe non-house images may still upload; quality warnings only
Region UX	Brush/erase yes; full redraw/rename/delete region polish is limited

6. Assumptions that can fail

- Shot is approximately frontal
- Building is low-rise residential exterior
- Masks after edit represent surfaces that will actually be finished
- User rates reflect local market

If these fail, both visualization quality and cost credibility degrade.

7. Honest positioning

Strength	Bound
Fast visual “what if” on the user’s own photo	Not a construction drawing
Transparent qty × rate math	Not a surveyor’s BOQ

Editable rates + PDF for contractor talks	Not a signed quotation
---	------------------------

The problem statement asked for **reasonably accurate** area for costing purposes. This prototype meets that bar when the user calibrates facade size and corrects masks; it does **not** claim centimetre survey accuracy.

06 — API Reference

Base URL (local): `http://localhost:8000`
API prefix: `/api`
Interactive docs: `http://localhost:8000/docs`

Health & static

Method	Path	Notes
GET	<code>/health</code>	Liveness + backend meta
GET	<code>/storage/...</code>	Static serving of uploads, masks, outputs, textures

Ingestion

Method	Path	Body	Response
POST	<code>/api/ingestion/upload</code>	multipart/form-data field <code>file</code>	<code>project_id</code> , image URL, size, warnings
GET	<code>/api/ingestion/projects/{project_id}</code>		Project metadata

Rejects: unreadable image, wrong type, oversize, below minimum resolution.
Warns (still accepts): blurry, too dark, too bright.

Segmentation

Method	Path	Body	Response
POST	<code>/api/segmentation/{project_id}</code>	<code>{ "categories": string[], "model"?:"grounded_sam" }</code>	RegionMap list (masks, polygons, areas)
GET	<code>/api/segmentation/{project_id}/regions</code>		Current RegionMap
PUT	<code>/api/segmentation/{project_id}/regions/{category_id}</code>	Masked image payload (edited mask)	Updated region

Empty `categories` → defaults: wall, balcony, rooftop, gate, window, door.

Materials & rates

Method	Path	Body	Response
GET	/api/materials	—	Catalog (paint + textures, rates, units)
PUT	/api/materials/rates	Rate update list	Confirmation

Meta

Method	Path	Response
GET	/api/meta/categories	Default + full category taxonomy
GET	/api/meta/models	Available segmentation backends
POST	/api/meta/models/activate	Activate model key
GET	/api/meta/render-modes	Available render modes (classical)
GET	/api/meta/model-status	Download / readiness of weights

Rendering

Method	Path	Body	Response
POST	/api/rendering/{project_id}	{ "selections": [{ category, material_key, color? }], "mode"? : "classical" }	input_url, output_url, applied list

Requires segmented regions for selected categories.

Costing

Method	Path	Body	Response
POST	/api/costing/{project_id}/estimate	{ selections, facade_width_m, facade_height_m, optional waste }	Lines, subtotal, total INR, disclaimer

See [04_Estimation_and_Costing.md](#) for formulas.

Reporting

Method	Path	Body	Response
POST	/api/reporting/{project_id}/pdf	selections + facade dims (same spirit as costing)	PDF download

Requires a prior successful render (`output_url` present).

Typical error codes

Code	Meaning
404	Unknown project
422	Unusable upload, missing region, unknown material, empty selections, no render for PDF

RegionMap fields (serialization)

Field	Description
category	Facade label
mask_url	Tinted RGBA preview
polygons	Contours for overlays
pixel_area	Mask pixel count
instance_count	Merged detections
confidence	Mean detector score

07 — Setup & Deployment

Prerequisites

- Python 3.11+ recommended
- Node.js 20+ / npm
- ~8 GB RAM preferred for Grounded SAM on CPU
- Disk space for model weights under `backend/storage/models/`

1. Backend (local)

```
cd "C:\Users\palak\Desktop\E2M\AI Exterior Studio\backend" python -m venv .venv
.\.venv\Scripts\Activate.ps1 # CPU torch first, then requirements pip install torch==2.5.1
torchvision==0.20.1 --index-url https://download.pytorch.org/whl/cpu pip install -r
requirements.txt # Optional: procedural textures if catalog folders are empty python
scripts/generate_textures.py # One-time model download (Grounding DINO + SAM) python
scripts/download_models.py copy .env.example .env # if needed uvicorn app.main:app --reload
--port 8000
```

- API: `http://localhost:8000`
- OpenAPI: `http://localhost:8000/docs`

2. Frontend (local)

```
cd "C:\Users\palak\Desktop\E2M\AI Exterior Studio\frontend" npm install copy
.env.local.example .env.local # Ensure NEXT_PUBLIC_API_BASE=http://localhost:8000 npm run
dev
```

- App: `http://localhost:3000`

3. Docker Compose (recommended)

The app is **fully Dockerized**. Prefer this path so frontend and backend run together without separate Python/Node setup.

From project root:

```
cd "C:\Users\palak\Desktop\E2M\AI Exterior Studio" docker compose up --build
```

Then access the platform:

Service	URL
---------	-----

Platform (UI)	http://localhost:3000
Backend API docs	http://localhost:8000/docs

Compose mounts `./backend/storage` into the backend so uploads, textures, and models stay on the host.

Note: Download models once with `python scripts/download_models.py` into `backend/storage/models/` before relying on segmentation in Docker. Host machine should have about **8 GB RAM** available for the backend container.

4. Environment highlights

Backend (.env)

Variable	Role
<code>CORS_ORIGINS</code>	Allow frontend origin
<code>DEFAULT_SEGMENTATION_MODEL</code>	grounded_sam
<code>MODEL_CACHE_DIR</code>	storage/models
<code>LOG_LEVEL</code>	Logging verbosity

Frontend

Variable	Role
<code>NEXT_PUBLIC_API_BASE</code>	Backend origin (baked at build time for Docker)

5. Smoke test

1. Open `http://localhost:3000`
2. Upload a house exterior photo
3. Click Detect Elements (wait for masks)
4. Assign a paint colour → Apply finishes & render
5. Confirm before/after and cost panel
6. Download PDF

6. Folder map (runtime)

```
backend/storage/ uploads/ # ingested photos masks/ # region overlays outputs/ # rendered
redesigns textures/ # cladding / tiles / patterns models/ # Grounding DINO + SAM weights
brand/ # report logo assets (if present)
```

7. Cloud note (Railway)

The project includes Railway config (backend/railway.toml, frontend/railway.toml) and is **fully Dockerized**, so cloud deploy was evaluated.

Why Railway deployment was not completed

Factor	Detail
Railway free trial	Typically about 1 GB RAM per service
This app's need	Grounding DINO + SAM on CPU need roughly 8 GB RAM to load models and run segmentation reliably
Outcome	Services may start, but model load / Detect Elements fails or is killed under memory pressure

So we **do not proceed with Railway hosting** on the free tier. A paid plan (or any host) with $\geq$ **8 GB RAM** for the backend would be required before cloud deploy is practical.

What to use instead: run the Docker containers on a local machine (or any VM) that has enough RAM:

```
cd "C:\Users\palak\Desktop\E2M\AI Exterior Studio" docker compose up --build
```

Then open the platform at **http://localhost:3000** (API docs at <http://localhost:8000/docs>).

The codebase remains cloud-ready in structure (Dockerfiles + Compose + Railway toml). Scaling to Railway later only needs an instance size that can hold the ML models in memory.